

SK Networks Family AI Camp 20기 3차 프로젝트

창업자를 위한 지식재산(IP)·지원정책 RAG 기반 챗봇 시스템

공식 문서 기반의 Multi-RAG 아키텍처로
신뢰할 수 있는 창업 지원 정보와 법령 해석을 제공합니다.



거침없이 RAG킵



김태빈



정래원



최소영



최유정

2025.12.10 ~ 2025.12.11

Table of Contents

창업자를 위한 IP·지원정책
RAG 챗봇 시스템 개발기



01
소개 (Intro)



03
데이터 처리



05
RAG 엔진



07
UI 계획 (Streamlit)



09
한계점 및 개선 방향



02
기술 아키텍처



04
Embedding · VectorDB



06
프롬프트 전략



08
데모 시나리오



10
결론 및 Q&A

Project Overview

프로젝트 개요 및
핵심 기술 정의



개발 목적 (Objectives)

창업자가 직면하는 정보의 파편화 문제를 해결하기 위해, **IP(지식재산)** · **정책** · **지원사업** · **실패사례** 데이터를 통합하여 신뢰할 수 있는 원스톱 질의응답 시스템을 구축합니다.



핵심 기술 파이프라인



Chunking



Embedding



ChromaDB



Transform
Query



Multi RAG



근거기반
답변



주요 산출물 (Deliverables)



전처리 완료된 Chunk Data



문서별 독립 VectorDB



LangChain RAG 파이프라인



Streamlit UI 프로토타입

💡 개발 동기

창업 생태계 현황 (2024년)

경기 둔화와 3고(高) 현상으로 창업 생태계가 위축되었습니다.

전체 창업기업: **118만 2,905개**

전년 대비 증감률: **4.5% 감소** (55,712개 ↓)

⚠ 기존의 문제점

정보 파편화: 기관별로 산재된 지원사업 정보

법령 난이도: 일반인이 해석하기 어려운 창업지원법

비교 불가: 수많은 지원사업의 통합 비교 어려움

개발 목표 (Solution)

공식 문서 기반 RAG 챗봇 시스템

창업자가 필요로 하는 IP-정책 정보를 **정확하고 신뢰할 수 있는** 근거를 바탕으로 제공합니다.

중소벤처기업부(장관 오영주)는 28일(금) '2024년 창업기업동향'을 발표했다.

2024년 전체 창업은 국내의 경기둔화와 3호(고금리·고환율·고물가) 지속에 따른 소비위축, 저출산·고령화 지속 등이 창업에 부정적으로 작용하여 전년대비 4.5%(55,712개) 감소한 **118만 2,905개***로 집계되었다.

* 연간 창업기업(개): ('20) 1,484,667 → ('21) 1,417,973 → ('22) 1,317,479 → ('23) 1,238,617 → ('24) 1,182,905

업종별로 살펴보면 개인간병 서비스, 건물관리 및 청소 등의 수요 증가, 숲속커머스·OTT·리테일미디어 등 광고시장의 패러다임 변화(채널 다양화)와 기저효과 등으로 개인 서비스업(8.0%↑), 전문·과학·기술(2.5%↑), 사업시설관리(3.0%↑) 등은 전년대비 신규창업이 증가한 것으로 나타났다.

* 증가업종: 개인서비스(8.0%↑), 전문·과학·기술(2.5%↑), 사업시설관리(3.0%↑), 전기·가스·공기(2.2%↑), 운수·창고업(0.5%↑), 교육 서비스업(0.6%↑), 농·임·어업 및 광업(0.1%↑), 금융·보험업(4.9%↑), 보건·사회복지(3.9%↑), 수도·하수·폐기(14.8%↑)

반면, 고금리, 부동산 침체, 원자재 가격 상승, 내수둔화 등의 영향으로 도 · 소매업(7.1%↓), 숙박 · 음식점업(7.7%↓), 부동산업(8.6%↓) 등에서 창업이 감소했다.

* 감소업종: 도·소매업(7.1%↓), 숙박·음식점업(7.7%↓), 부동산(8.6%↓), 정보통신업(9.8%↓), 제조업(5.9%↓), 예술·스포츠·여가(6.1%↓), 건설업(1.7%↓)

월별로 볼 때, 전체창업은 대부분 月에서 전년동월대비 감소했으나 1월, 4월, 7월, 10월, 12월은 소폭 증가하는 모습도 보였다.

< 월별 창업기업 수 및 증감률 >

(단위 : 개, %)												
구분	1월	2월	3월	4월	5월	6월	7월	8월	9월	10월	11월	12월
전체	121,183	98,207	105,602	104,899	99,807	93,062	105,709	91,490	82,631	99,378	90,208	90,729
	(14.6)	(△10.2)	(△10.7)	(3.4)	(△7.2)	(△13.9)	(1.2)	(△12.1)	(△10.1)	(0.01)	(△12.1)	(5.9)
* () : 전년동기대비 증감률												

Project Necessity

왜 이 서비스가 필요한가요?

SKN20-3rd-4TEAM



난해한 공문서·법령의 장벽 해소

전문 용어와 복잡한 문장으로 구성된 PDF 및 법령 원문은 초기 창업자가 스스로 해석하기 어렵습니다.

→ 평이한 설명과 정확한 근거 인용을 통해 정보 접근성 향상



흩어진 지원 정보의 통합

기관별로 산재된 창업 지원 사업 정보는 일일이 찾아다녀야 하며, 조건 비교가 매우 번거롭습니다.

→ 한 곳에서 검색·요약·비교 가능한 통합 플랫폼 제공

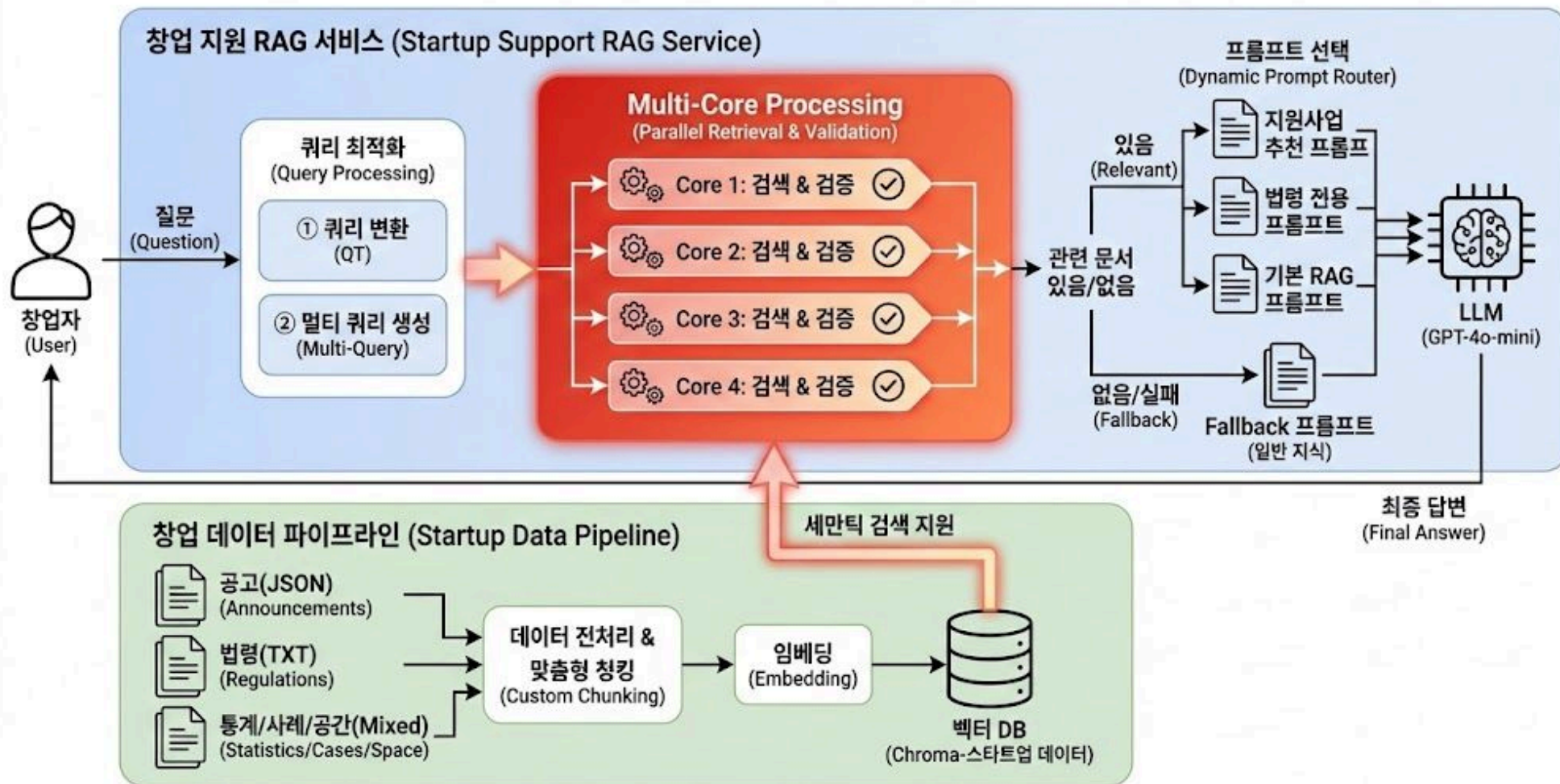


법적 리스크 및 환각(Hallucination) 방지

일반 LLM의 부정확한 정보 제공은 창업자에게 치명적인 법적/재정적 리스크를 초래할 수 있습니다.

→ RAG 기술로 공식 문서를 근거로 제시하여 신뢰성 확보

창업 지원 통합 AI 어시스턴트





통합 RAG 데이터 구조

거침없이 RAGkick



단일 ChromaDB 통합 구성

모든 문서를 하나의 벡터 저장소에 통합하고, **metadata**를 활용하여 효율적으로 관리합니다.



announcement

지원사업 공고 (K-Startup)



law

중소기업창업 지원법



cases

실패 및 재창업 사례



space

창업 공간 정보



stat

창업 통계 및 연구 자료



program_chunk

지원 프로그램 상세



ip_manual_chunk

지식재산 관리 매뉴얼



구조 변경 이점: 기존 독립 DB 방식 대비 관리가 용이하며, **Metadata Filtering**을 통해 필요한 문서 타입만 타겟팅하여 검색 정확도를 높였습니다. Hybrid Search 적용 시 전체 풀에서 키워드 검색이 가능합니다.



Document 변환 로직 (JSON → Document)

SKN20-3rd-4TEAM

⚙️ 핵심 로직 설명

데이터 정규화: 다양한 JSON 필드를 RAG 검색에 최적화된 텍스트 포맷으로 재구성

메타데이터 태깅: 필터링(지역, 날짜, 유형)을 위한 핵심 메타데이터 data_type, source 부여

HTML 정제: clean_html 함수로 불필요한 태그 제거

🗄️ 주요 함수

create_announcement_document

create_stat_document

create_space_document

```
src/preprocess/main_chunking.py

56 def create_announcement_document(item):
57     """JSON 항목을 LangChain Document 객체로 변환"""
58
59     # 1. 검색에 사용될 본문 텍스트 구성 (사람이 읽기 좋은 형태)
60     page_content = f"""
61     【사업 공고】 {item.get('biz_pbanc_nm', '제목 없음')}
62
63     ■ 사업 개요
64     {clean_html(item.get('pbanc_ctnt', ''))}
65
66     ■ 신청 대상
67     - 대상 유형: {item.get('aply_trgt', '')}
68     - 접수 기간: {format_date(item.get('pbanc_rcpt_bgng_dt'))} ~
69     {format_date(item.get('pbanc_rcpt_end_dt'))}
70     """.strip()
71
72     # 2. 필터링 및 출처 추적을 위한 메타데이터 부여
73     metadata = {
74         "source": "k-startup",
75         "data_type": "announcement", # 문서 타입 (라우팅용)
76         "biz_id": item.get('pbanc_sn'),
77         "category": item.get('supt_biz_clsfc', ''),
78         "region": item.get('sunt_regin', ''),
```



main_chunking.py 구조

거침없이 RAGkick

 JSON 데이터
dataset.json

 TXT 파일
법령, 사례 등

 TXT 폴더
지원프로그램 등



Document 변환 & 로딩

create_announcement_document()
TextLoader / DirectoryLoader



RecursiveCharacterTextSplitter

타입별 최적화 설정 적용
(Type-specific Configs)



Output Pickle

chunked_documents.pkl

🔧 핵심 유틸리티

- **format_date** : 8자리 날짜(20251201) → 표준(2025-12-01) 변환
- **clean_html** : 정규식으로 불필요한 태그/공백 제거 및 정제

📄 Document 생성 로직

- **JSON**: announcement, stat, space 별 메타데이터 매핑
- **TXT**: **TextLoader** (단일), **DirectoryLoader** (폴더) 활용
- **통합**: 모든 문서를 LangChain **Document** 객체로 표준화

📊 타입별 청킹 최적화 (Chunk / Overlap)

📖 Law (법령)

700 / 100

📑 Cases (사례)

500 / 80

📢 Announcement

400 / 80

📊 Stat (통계)

200 / 30

⚠️ **Skip**: Program/Manual Chunk (기존 청킹 유지)

📦 최종 산출물

전처리된 Document 리스트를 **chunked_documents.pkl** 로 저장 (VectorDB용)

타입별 최적화 Chunking 전략

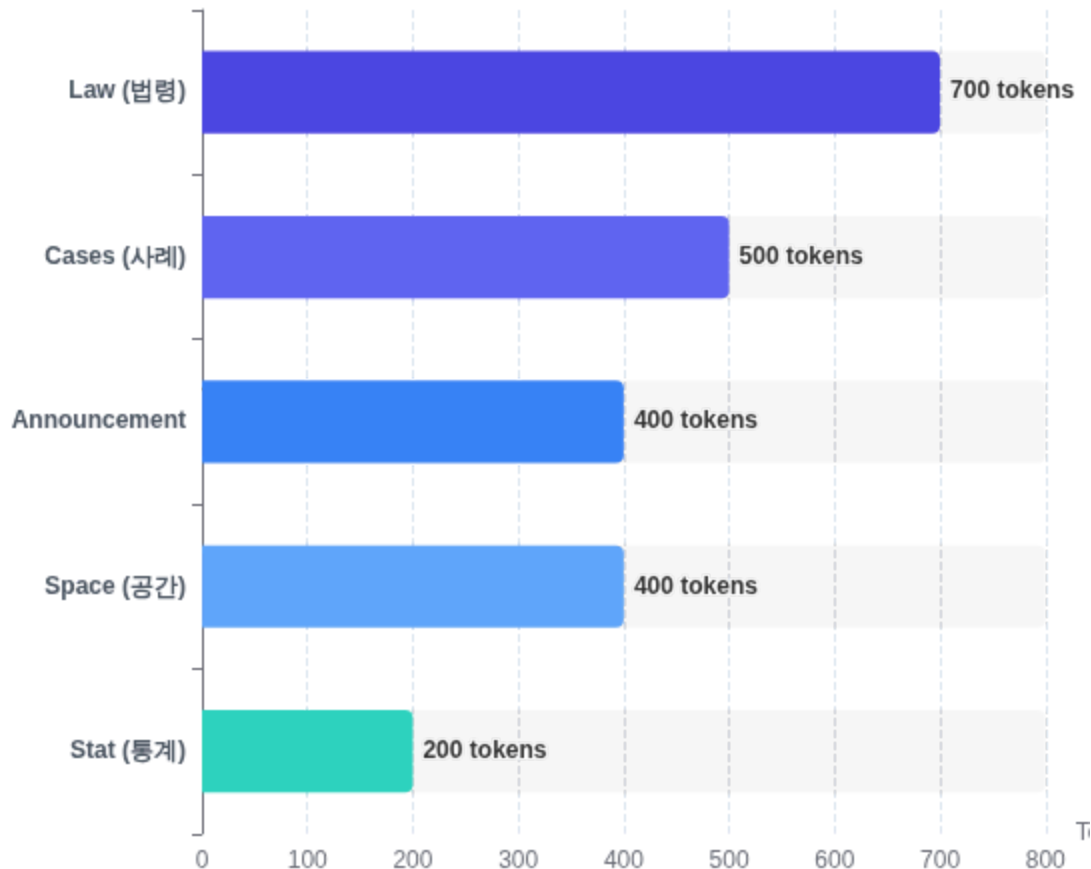
Data Type Configuration

Data Type	Chunk Size	Overlap	Rationale
Law (법령)	700	100	조/항 단위 맥락 유지
Cases (사례)	500	80	스토리 흐름 보존
Announcement	400	80	지원 요건 단위
Space (공간)	400	80	시설 정보 그룹화
Stat (통계)	200	30	핵심 수치 중심

Optimization Impact

- ✓ 기존 일괄 전략(300/50) 대비 문서 특성 반영
- ✓ 법령의 경우 긴 Context Window를 활용하여 법적 해석 오류 방지
- ✓ 통계 데이터는 짧은 단위로 잘라 검색 정밀도(Precision) 강화

Chunk Size Comparison by Data Type



Embedding Strategy

벡터 임베딩 모델 선정 및
인덱싱 최적화 전략

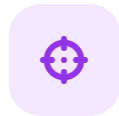


text-embedding-3-small

MAIN MODEL

속도와 비용 효율성이 최적화된 모델로, 대량의 문서 처리에 적합합니다.

- ✓ 빠른 응답 속도 (Low Latency)
- ✓ 저렴한 임베딩 비용



text-embedding-3-large

ALTERNATIVE

복잡한 문맥 파악이 필요하거나 더 높은 정확도가 요구될 때 사용합니다.

- ✓ 높은 의미적 정확도
- ✓ 법령 등 정밀 해석 필요 시 활용



Indexing Strategy

OPTIMIZATION

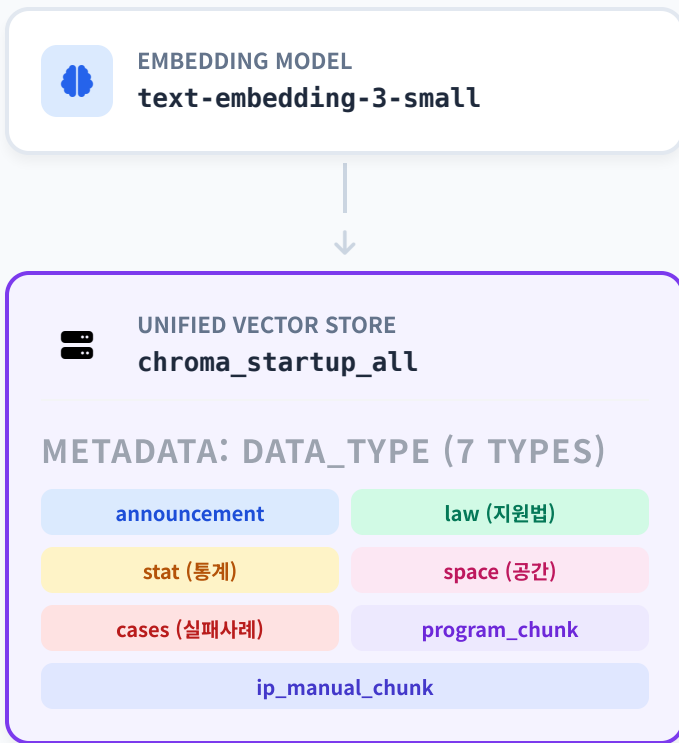
문서 타입별 독립 인덱스와 메타데이터 필터링을 병행합니다.

Manual DB

Strategy DB

Law DB

Program DB



단일 컬렉션 + 메타데이터 구조

☰ 단일 DB 통합 및 메타데이터 활용

- ✓ **단일 DB 통합 (Unified Architecture):**
기존 5개 독립 DB 구조에서 chroma_startup_all 단일 경로로 통합하여 관리 포인트 최소화 및 검색 효율성 증대
- ▼ **메타데이터 필터링 (Metadata Filtering):**
7가지 data_type을 메타데이터로 부여하여, 필요 시 특정 문서 유형만 타겟팅하거나 전역 검색(Global Search) 유연하게 수행

```
vectorstore = Chroma(  
    persist_directory="./chroma_startup_all",  
    collection_name="startup_all_rag", # 단일 컬렉션  
    embedding_function=embedding_model  
)  
# Metadata Example: {"data_type": "law"}
```

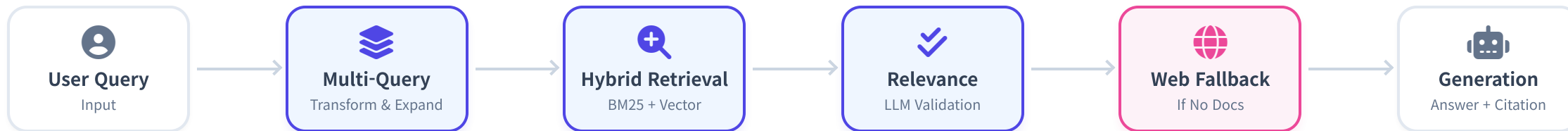
Optimization

↑ 통합 구조로 전환하여 **Hybrid Search** 및 **Multi-Query** 적용 시 복잡한 라우팅 로직 없이 일관된 검색 성능 확보



RAG 엔진 워크플로우

거침없이 RAG킹



01 쿼리 최적화 & 확장

Query Transform Multi-Query

사용자 질문을 핵심 키워드로 변환(Query Transformation)하고, 3가지 다른 관점의 질문(Multi-Query)을 생성하여 검색의 리콜(Recall)을 극대화합니다.

02 하이브리드 검색 & 검증

BM25 + Vector LLM Judge

키워드(BM25)와 의미(Vector) 검색을 결합하여 정확도를 높이고, 검색된 문서가 질문과 실제 관련이 있는지 LLM을 통해 엄격하게 검증(Relevance Checking)합니다.

03 폴백(Fallback) & 생성

Tavily Search GPT-4o-mini

내부 문서에 관련 정보가 없을 경우 자동으로 웹 검색(Tavily API)으로 전환(Fallback)하며, 확보된 최적의 정보를 바탕으로 근거(Citation)를 포함한 답변을 생성합니다.



Query Transformation (검색 최적화)

SKN20-3rd-4TEAM

🔍 검색어 최적화 전략

키워드 압축: 자연어 질문의 노이즈(서술어, 인사말 등)를 제거하고 검색 엔진 친화적인 핵심어 추출

중심 키워드: 법령·지원대상·자격요건 등 도메인 특화 용어 위주로 재구성

LCEL 적용: Prompt | LLM | Parser 구조

📈 기대 효과

Recall 향상: 구어체 표현("돈 좀 빌리고 싶은데")을 정책 용어("자금 융자 지원")로 변환

노이즈 감소: 불필요한 단어로 인한 엉뚱한 문서 검색 방지

src/rag/query_transformation.py

```
125 # 5. Query Transformation (검색용 질문 변환)
126 query_transform_prompt = ChatPromptTemplate.from_template(
127     """
128     다음 질문을 '중소기업창업 지원법' 검색에 가장 적합한
129     핵심 키워드 형태로 변환하세요.
130     불필요한 말은 제거하고,
131     법령·지원·대상·요건 중심 키워드로 만들어 주세요.
132     한 줄로만 출력하세요.
133
134     원본 질문: {question}
135     변환된 검색어:
136     """
137 )
138
139 query_transform_chain = (
140     query_transform_prompt
141     | llm
142     | StrOutputParser()
143 )
144
145 # 실행 예시
```

Retriever 설계

Custom Multi-Query
Vector Retrieval

Not `LangChain .as_retriever()`

거침없이 RAG키



① 내부 문서 검색: Custom Multi-Query Retriever

`vectorstore.similarity_search_with_score` 직접 구현



Query
Transform



Multi-Query
생성



Vector Search
(Score)



유사도 필터링
(≥ 0.3)



LLM
Relevance

Bing/Google 처럼 질의를 여러 번 확장하여 최적의 문서를 검색. 유사도 점수 기반 1차 필터링 후, LLM이 실제 관련성을 최종 판단하여 노이즈를 제거합니다.



② Tavily Search API

Fallback 1

조건: 내부 문서 관련성 부족 시

외부 검색 결과를 Document로 변환하여 RAG 엔진에 주입, 최신/외부 정보를 보완합니다.



③ Fallback LLM Only

Fallback 2

조건: 내부 문서 X & 웹 검색 실패

검색 결과가 없을 경우, LLM의 일반 상식에 기반하여 답변을 생성합니다.



멀티쿼리 RAG + 웹 검색 Fallback

거침없이 RAG 킷

✓ 이중 필터링 (Dual Filtering)

1차 유사도 필터: Distance 기반으로 0.3 미만 문서 제거

2차 LLM 검증: `check_relevance` 함수로 문서 내용이 질문과 실제 관련있는지 LLM이 최종 판단

🌐 웹 검색 Fallback 전략

내부 데이터 부족 시: 내부 문서가 없거나 관련성이 낮다고 판단되면 즉시 외부 검색 전환

Tavily API 연동: 최신 정보를 웹에서 실시간으로 수집하여 답변 생성 (RAG Scope 확장)

src/rag/rag_base_multi_web.py

```
35 def multi_query_rag_with_qt(question, top_k=10):
36     # 1. 쿼리 변환 & 멀티쿼리 벡터 검색
37     qt_query = qt_chain.invoke({"question": question})
38     queries = multi_query_chain.invoke({"question": qt_query}).splitlines()
39     all_docs = search_documents(queries)
40
41     # 2. 유사도 필터링 & LLM 관련성 검증
42     filtered_docs = filter_by_similarity(all_docs, threshold=0.3)
43     is_relevant = False
44     if filtered_docs:
45         is_relevant = check_relevance(question, filtered_docs)
46
47     # 3. 최종 분기: 내부 RAG vs 웹 검색 (Fallback)
48     if is_relevant:
49         # 내부 문서가 충분하면 내부 RAG 실행
50         return rag_answer_from_docs(question, filtered_docs[:top_k])
51     else:
52         # 내부 문서 없거나 무관함 → 웹 검색 시도 (Tavily)
53         web_docs = web_search(question)
54         if web_docs:
55             return rag_answer_from_docs(question, web_docs)
56         else:
57             return fallback_chain.invoke({"question": question})
```



Dynamic Prompt Routing (프롬프트 라우팅)

거침없이 RAG하기

라우팅 메커니즘 (Routing Logic)



INPUT ANALYSIS

사용자 질문의 핵심 키워드 및 의도 파악



KEYWORD MATCHING

정의된 키워드 셋과 매칭 (추천/법령/일반)



PROMPT SELECTION

최적의 시스템 프롬프트(Persona) 연결



프롬프트 유형 (Personas)

지원사업

신청 자격, 지원 혜택, 마감 기한 중심 추천

법령해석

조문 근거(제n조) 기반의 엄격한 법리 해석

일반 RAG

포괄적인 창업 정보 및 일반 상담 제공

src/rag/prompt_router.py

```
1 def choose_prompt(question: str):
2     # 1. 의도 분류를 위한 키워드 정의
3     recommend_keywords = [
4         "추천", "맞는", "신청할 수 있는", "지원해주는",
5         "사업 알려줘", "혜택", "지원금"
6     ]
7     law_keywords = [
8         "정의", "자격", "요건", "지원법",
9         "법에서", "법상", "제도", "규정"
10    ]
11
12    # 2. 키워드 매칭 및 프롬프트 선택
13    if any(k in question for k in recommend_keywords):
14        return recommend_prompt, "recommend_prompt"
15
16    if any(k in question for k in law_keywords):
17        return law_prompt, "law_prompt"
18
19    # 3. 기본값 (General RAG)
20    return rag_prompt, "rag_prompt"
21
22    # 실행 예시
```



프롬프트 엔지니어링 전략 (1/2)

거침없이 RAG 킷

```
prompts/basic_rag_prompt.py

84 # 기본 RAG 프롬프트
85 rag_prompt = ChatPromptTemplate.from_messages([
86     ("system", """
87     당신은 예비·초기 창업자를 도와주는 '창업 지원 통합 AI 어시스턴트'입니다.
88
89     [사용 가능한 정보 유형]
90     - 지원사업 공고 (announcement)
91     - 실패/재도전 사례 (cases)
92     - 창업 공간 정보 (space)
93     - 법령: 중소기업창업 지원법 등 (law)
94     - 통계, 매뉴얼 등 참고 자료
95
96     [답변 원칙]
97     1. 반드시 제공된 문맥(Context) 안의 정보만 사용하세요.
98     2. 문맥에 없는 내용은 추측하지 말고 솔직하게 말하세요.
99     3. 질문 성격에 따라 다음 정보 유형을 우선 활용하세요.
100     - 지원사업·신청 가능 여부 → announcement
101     - 법적 정의·자격 요건 → law
102     - 조언·주의점 → cases
103     - 공간·입주 → space
104     4. 핵심 답변 후 필요하면 bullet로 정리하세요.
105     5. 마지막에 참고 근거 유형을 요약하세요.
106     """),
107     ("human", "[문맥]\n{context}\n\n[질문]\n{question}\n\n[답변]")
```

1 기본 RAG 프롬프트

7가지 데이터 유형(공고, 사례, 공간 등)을 종합적으로 활용하며, 답변 원칙 (Fact-based) 준수와 출처 명시를 강제하여 환각 현상을 제어합니다.

ROLE PLAYING

SOURCE CITATION

ANTI-HALLUCINATION

```
prompts/recommendation_prompt.py

# 지원사업 추천 프롬프트
recommend_prompt = ChatPromptTemplate.from_messages([
    ("system", """
    당신은 예비·초기 창업자에게 가장 적합한 '지원사업을 추천하는 전문가 AI'입니다.

    [목표]
    사용자의 조건(나이, 지역, 업종, 창업 단계 등)을 기준으로
    '실질적인 도움이 되는 사업(자금·공간·R&D·시제품·교육)'을 우선적으로 추천합니다.

    [추천 우선순위]
    1. 현금성 지원(사업화 자금, 시제품 제작비, R&D)
    2. 입주 공간, 장비 지원
    3. 액셀러레이팅, 멘토링
    4. 단순 교육/특강은 마지막 순위

    [추천 규칙]
    1. 반드시 announcement 문서만 사용
    2. 사용자 조건과 '지역·연령·단계·업종'이 명확히 맞는 것만 추천
    3. 최대 2개까지만 추천
    4. 조건이 맞는 사업이 없으면 솔직하게 말하기
    5. IT·서비스업이면 '기술·콘텐츠·플랫폼' 키워드 포함 사업 우선

    [출력 형식]
    ■ 추천 사업명
    ■ 왜 이 사용자에게 적합한지
    ■ 지원 내용(자금/공간/교육 중 무엇인지 명확히)
    """)
```

2 지원사업 추천 프롬프트

사용자의 상황(업력, 지역, 분야)을 분석하여 최적의 사업을 선별합니다. 우선 순위 규칙(자금 > 공간 > 멘토링)을 통해 실질적 혜택 중심의 추천을 제공합니다.

PRIORITY RULE

LOGIC CHAIN

USER CONTEXT



```
prompts/law_specialist.py

# 법령 전용 프롬프트
law_prompt = ChatPromptTemplate.from_messages([
    ("system", """
당신은 중소기업창업 지원법을 바탕으로 창업 제도와 요건을 설명하는 AI입니다.

[규칙]
1. 반드시 문맥에 있는 법령 내용만 사용하세요.
2. 가능하면 조문 번호(제o조)를 함께 제시하세요.
3. 문맥에 없는 내용은 "제공된 법령 문서에서 해당 내용은 확인되지 않습니다."라고 답하세요.
4. 답변 끝에 "* 본 답변은 일반 정보 제공이며, 구체적인 법률 자문은 아닙니다."를 포함하세요.
"""),
    ("human", "[법령 문맥]\n{context}\n\n[질문]\n{question}\n\n[설명]")
])
```

```
prompts/fallback_handler.py

# Fallback 프롬프트
fallback_prompt = ChatPromptTemplate.from_template("""
질문: {question}

내부 문서에서 관련 정보를 찾지 못했습니다.
일반적인 지식을 바탕으로 답변해주세요.

답변: """])
```

3 법령 전용 프롬프트

'중소기업창업 지원법' 등 법적 근거에 기반하여 엄밀하게 답변합니다. 조문 번호(제 O조)를 명시하고, 법률 자문이 아님을 알리는 면책 조항을 포함합니다.

- LEGAL BASIS
- DISCLAIMER
- STRICT CONTEXT

4 Fallback 프롬프트

내부 문서 검색 결과가 없을 때 작동합니다. 일반 상식으로 대응하거나 Tavily 웹 검색 결과 활용을 유도하여 '답변 실패' 경험을 방지합니다.

- EXCEPTION HANDLING
- WEB KNOWLEDGE
- UX SAFETY

근거 기반 답변 & 출처 추적

Evidence-based Answers
& Source Tracking Strategy

🔍 RAG 신뢰성 확보를 위한 핵심 메커니즘



Context 구성 전략

단순 텍스트뿐만 아니라 파일명(Source) 메타데이터를 함께 주입하여 LLM이 출처를 인지하도록 구성합니다.

```
Context = "문서 본문" + f"(Source: {doc.metadata['source']})"
```



출처 표기 (Citation)

사용자가 답변의 근거를 즉시 확인할 수 있도록 파일명을 명시합니다.

- 답변 하단에 `sources` 리스트 자동 생성
- `os.path.basename` 을 사용하여 경로 제외, 깔끔한 파일명 출력



운영 정책 (Policy)

할루시네이션(Hallucination) 방지와 정보의 정확성을 위한 엄격한 규칙을 적용합니다.

🚫 문서 혼합 금지 (Routing)

✅ Chunk 일관성 유지

🖥 UI 구조 계획 (Streamlit)

📦 Streamlit 최신 기능 적용

- **웹 검색 출처:** 외부 Tavily 검색 결과도 명확히 표시
- **대화 히스토리:** 세션 내 질의응답 기록 관리 및 조회
- **출처 필터링:** 문서 유형별 출처 태그 시각화

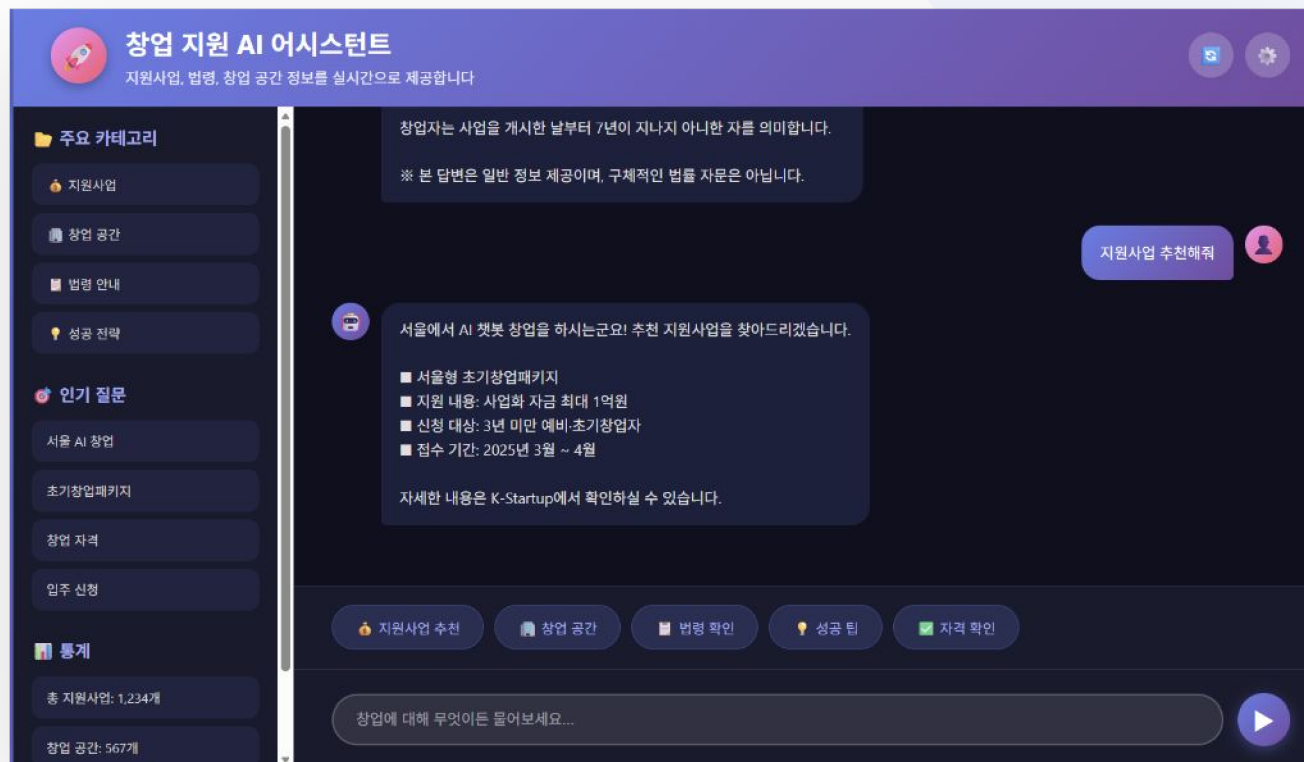
🔄 서비스 워크플로우

1. 사용자 질의 입력 (추천 질문 활용 가능)
2. 질의 분석 및 Query Transformation 실행
3. 내부 문서/웹 검색 결과 기반 답변 생성
4. 답변과 함께 **내부 문서/웹 출처** 구분 표시

⚡ UX 및 편의성 최적화

사용자 경험 향상을 위한 최적화:

- st.status를 활용한 검색/생성 진행 상태 시각화
- 직관적인 출처 태그 UI로 신뢰도 강화





데모 시나리오

거침없이 RAGkick

☰ 주요 시나리오

법령/규정

창업자 자격 요건 문의

중소기업창업 지원법상 창업자로 인정받기 위한 법적 기준 및 예외 사항 질의

IP 전략

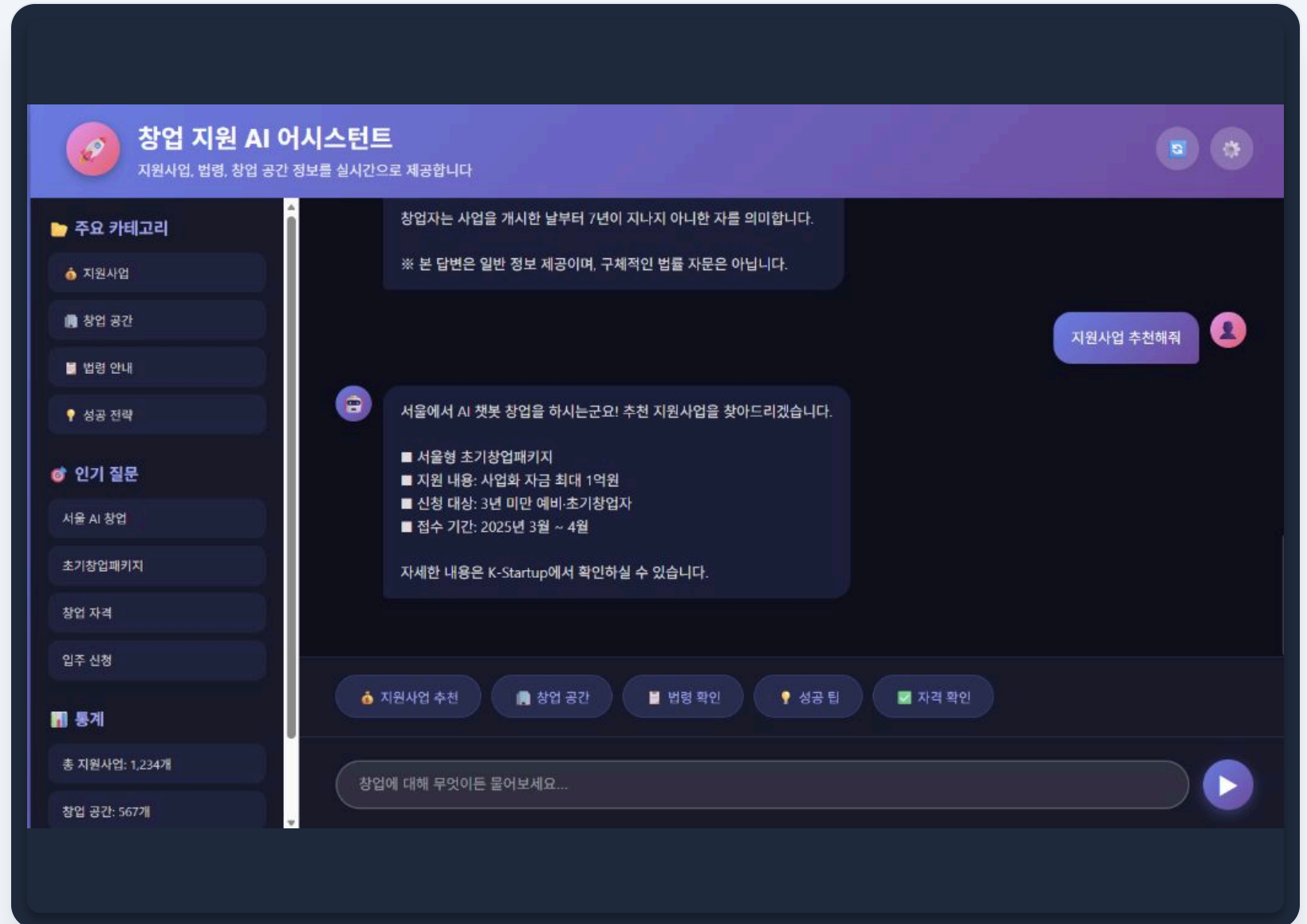
2025 지식재산 가이드

최신 IP 경영 전략의 핵심 변경점 요약 및 스타트업 적용 방안 확인

지원사업

맞춤형 지원사업 추천

사용자 상황(서울 소재, AI 분야)에 적합한 지원사업 추천 및 공고 내용 요약



Future Improvements

시스템 고도화 및
성능 평가 결과

- ✓ 평가 시스템 구축 완료
- ✓ 5개 모델 성능 비교
- ✓ HTML 리포트 자동화



정량적 평가 시스템 구축 완료

eval_compare.py를 통한 자동화된 성능 측정 및 HTML 리포트 생성

구현 완료 ✓

RAG 시스템 성능 비교 (종합 점수)



● rag_base: 75.7% (Baseline)

● rag_base_multi_web: 92.8% (Best) 🏆

● rag_base_multi: 91.3%

● rag_base_multi_web_llm: 92.3%

최고 성능 모델 (MULTI-WEB)

키워드 정확도

89.6%

LLM 평가 점수

80.0%

실행 시간

12.97s

⚠️ CACHE 시스템 현황

rag_base_multi_web_cache 모델(91.6%) 테스트 완료되었으나, 현재 운영 환경에서는 미사용 상태 (향후 도입 검토).

RAG 시스템 평가 리포트

생성 시간: 2025-12-10 11:35:31

rag_base

Baseline Model

종합 점수 75.7%

키워드 점수 95.8%

LLM 점수 55.6%

평균 실행 시간 6.83초

오류율 0.0%

rag_base_multi

Multi-Query

종합 점수 91.3%

키워드 점수 93.8%

LLM 점수 88.8%

평균 실행 시간 15.51초

오류율 0.0%

rag_base_multi_we...

BEST PERFORMANCE

종합 점수 92.8%

키워드 점수 89.6%

LLM 점수 80.0%

평균 실행 시간 12.97초

오류율 0.0%

rag_base_multi_web_c...

With Caching

종합 점수 91.6%

키워드 점수 87.5%

LLM 점수 85.6%

평균 실행 시간 11.03초

오류율 0.0%



결론 및 향후 계획

SKN20-3rd-4TEAM



신뢰성 높은 정보 제공

공식 문서와 법령에 기반한 **Multi-RAG 시스템**을 구축하여, 생성형 AI의 환각 (Hallucination) 문제를 최소화하고 **명확한 출처(Citation)**를 제공하는 창업 가이드를 실현했습니다.



모듈형 아키텍처 & 확장성

데이터 전처리, VectorDB 구축, RAG 엔진을 **독립 모듈로 설계**하여, 향후 새로운 법령이나 지원사업 데이터가 추가되더라도 유연하게 확장 가능한 시스템 기반을 마련했습니다.



향후 확장 가능성

초기 창업자 가이드



기업 정책 사내 챗봇



B2G 대국민 민원 서비스

Q & A

경청해 주셔서 감사합니다. 질문이 있으시면 말씀해 주세요.

SK Networks Family AI Camp 20기
3차 프로젝트 Team 4